

README

LLMOrchestrator

Standalone Go module for managing headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) with hybrid pipe+file communication protocol.

Overview

LLMOrchestrator provides a unified interface for spawning, managing, and communicating with multiple LLM-powered CLI agents. It supports real-time stdin/stdout pipe communication (JSON-lines protocol) and file-based artifact exchange.

Go Module: `digital.vasic.llmorchestrator`

Features

- **5 CLI Adapters:** OpenCode, Claude Code, Gemini, Junie, Qwen Code
- **Thread-safe Agent Pool:** Capability-based matching with `Acquire(ctx, requirements)`
- **Circuit Breaker:** Per-agent health monitoring (3 consecutive failures = unhealthy)
- **Hybrid Communication:** Pipe (real-time JSON-lines) + File (inbox/outbox/shared directories)
- **Response Parser:** Structured extraction of actions, issues, and JSON from raw LLM output
- **Security:** Path traversal protection, response length limits, command injection prevention

Quick Start

```
# Build
go build ./...
```

```
# Test
```

```
go test ./... -race -count=1
```

```
# Run (standalone)
```

```
go run cmd/orchestrator/main.go
```

Architecture

```
LLMOrchestrator/
├─ cmd/orchestrator/  # Standalone CLI entry point
├─ pkg/
│   └─ agent/        # Agent interface, AgentPool,
HealthMonitor, CircuitBreaker
│   └─ adapter/      # BaseAdapter + 5 CLI adapters
│   └─ protocol/     # PipeTransport (JSON-lines),
FileTransport (inbox/outbox)
│   └─ parser/       # ResponseParser, action/issue extraction
│   └─ config/       # .env loading, agent path resolution
├─ Upstreams/        # Multi-remote sync scripts
└─ docs/
```

Configuration

Copy `.env.example` to `.env` and configure agent paths and API keys.
See [USER_GUIDE.md](#) for details.

Testing

```
make test      # Run all tests with race detector
make fuzz     # Run fuzz tests
make cover    # Generate coverage report
make check    # Run vet + tests
```

License

Apache License 2.0 - see [LICENSE](#).